

Pod IP는 어디서 오는가

1. 쿠버네티스 네트워크 모델

1.1 네트워크 원칙

- 모든 Pod는 클러스터 전역에서 유일한 IP를 가진다.
- **Pod 간 통신은 NAT 없이 통신한다.**
- 노드의 에이전트(kubelet 등)는 그 노드의 모든 Pod와 통신할 수 있다.

세 문장이 말하는 것은 하나다. 모든 Pod가 하나의 평평한 네트워크에 있는 것처럼 보여야 한다. 포트 매핑도 주소 변환도 없이, Pod A가 Pod B의 IP를 그대로 찍으면 닿아야 한다. 두 Pod가 같은 노드에 있든 다른 노드에 있든 애플리케이션 코드는 차이를 느끼지 않아야 한다.

쿠버네티스는 네트워크를 어떻게 구현할지 정의하지 않는다. 지켜야 할 조건만 정의할 뿐, 구현은 CNI 플러그인에 위임한다.

1.2 NAT 금지 이유

Docker의 기본 브리지 네트워크는 컨테이너를 `172.17.0.0/16` 사설 대역에 두고 `-p 8080:80` 형태의 DNAT/MASQUERADE로 외부에 노출한다. 이 구조에서는 컨테이너가 인식하는 자기 주소와 외부에서 부르는 주소가 달라진다.

문제의 뿌리는 주소가 두 개가 된다는 것이다. 컨테이너는 자기를 `172.17.0.5` 로 알고 있지만 남들은 `호스트IP:8080` 으로 부른다. 그러면 "자기 주소를 남에게 알려주는" 모든 동작이 어긋난다.

깨지는 것	이유
서비스 등록 / 디스커버리	자기 IP를 레지스트리에 등록해도 그 주소로는 도달할 수 없다
mTLS 인증서 SAN 검증	인증서에 기재된 IP와 상대가 관측한 IP가 불일치
분산 트레이싱 / 접근 로그	모든 요청의 소스 IP가 게이트웨이 하나로 관측됨
소스 IP 기반 정책	NetworkPolicy와 방화벽 ACL이 무력화

SNAT을 거치면 수신 측에서 보이는 출발지는 항상 게이트웨이 주소 하나다. 누가 보냈는지를 IP로 구분하는 모든 장치가 동시에 무의미해진다. 로그를 봐도 범인을 특정할 수 없고, 정책을 걸어도 전부 통과하거나 전부 차단된다.

그래서 쿠버네티스는 NAT을 제거하고, 그 대신 Pod IP를 클러스터 전역에서 라우팅 가능하게 만들라는 요구를 CNI에 넘겼다. 물리 네트워크가 Pod IP를 전혀 모르더라도 어느 노드의

Pod들 서로의 IP에 직접 도달할 수 있어야 한다. **Flannel·Calico·Cilium의 차이는 이 요구를 구현하는 방식의 차이**다. 터널로 감싸든(VXLAN), 라우팅을 광고하든(BGP), eBPF로 우회하든 결과만 같으면 된다.

1.3 통신 유형

유형	담당	비고
컨테이너 ↔ 컨테이너 (같은 Pod)	커널 (netns 공유)	localhost
Pod ↔ Pod	CNI	NAT 없음
Pod ↔ Service	kube-proxy (또는 eBPF)	ClusterIP는 가상 IP
외부 ↔ Service	NodePort / LoadBalancer / Ingress	SNAT 발생 가능

- CNI와 kube-proxy는 서로 다른 유형을 담당한다. kube-proxy를 정지시켜도 Pod 간 통신이 그대로 동작하는 이유가 이것이다. 반대로 CNI가 망가지면 둘 다 죽는다.
- netns 두 개와 브리지 하나로 Pod 네트워크를 구성하면 kube-proxy 없이도 ping이 성립한다. 쿠버네티스 없이 `ip` 명령만으로 재현할 수 있다는 뜻이며, Pod ↔ Pod 통신이 커널 기능만으로 완결된다는 증거이기도 하다.

1.4 IP 대역

- **Pod CIDR**
 - Pod에 할당되는 대역
 - 노드별로 미리 분할하거나(host-local) IPAM이 동적으로 관리(Calico)
- **Service CIDR**
 - ClusterIP용
 - 어느 인터페이스에도 존재하지 않는 가상 대역
- **Node CIDR**
 - 노드의 실제 패브릭 주소
 - 물리 네트워크가 인지하는 유일한 대역

세 대역을 가르는 결정적 차이는 **물리 네트워크가 아는가**이다.

대역	물리 네트워크가 아는가	어떻게 도달하는가
Node CIDR	안다	그냥 라우팅

대역	물리 네트워크가 아는가	어떻게 도달하는가
Pod CIDR	대개 모른다	오버레이 터널 또는 라우트 광고
Service CIDR	모르고, 알 필요도 없다	노드를 벗어나기 전에 DNAT으로 사라진다

Service CIDR이 이상하게 느껴지는 이유는 IP처럼 생겼는데 소유자가 없기 때문이다. 하지만 이 주소는 **패킷이 노드를 떠나기 전에 반드시 다른 주소로 바뀐다**. 선 위를 흘러 다니는 일이 없으니 실재할 필요도 없다. 그래서 Service CIDR은 Node CIDR·Pod CIDR과 겹치지만 않으면 아무 대역이나 골라도 된다.

1.5 NAT이 유지되는 지점

NAT이 없다는 것은 **Pod ↔ Pod 구간**에 한정된 이야기다. 클러스터 경계를 넘는 순간 NAT은 다시 등장한다. 외부 네트워크는 Pod IP를 모르기 때문에 어쩔 수 없다.

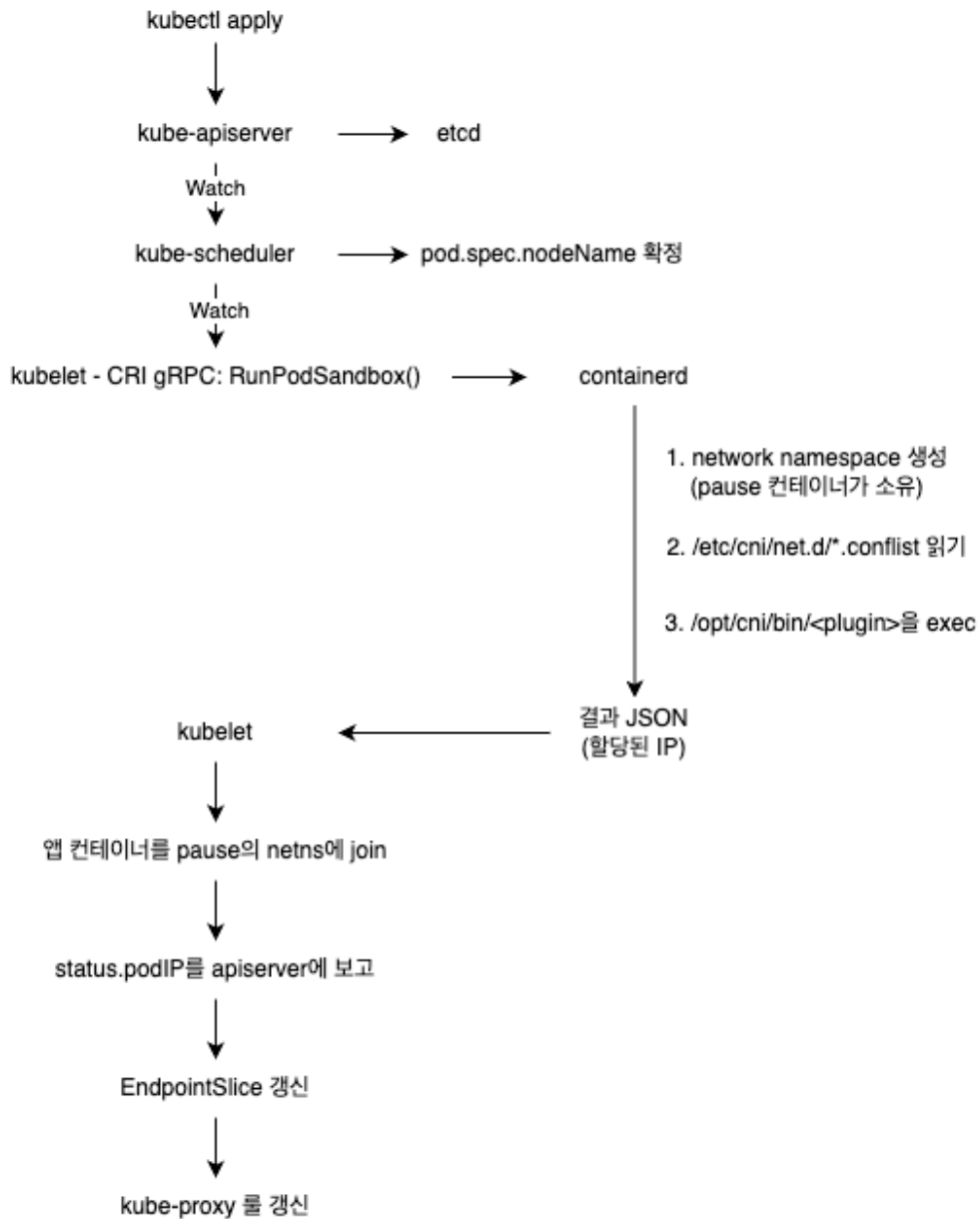
상황	동작
Pod → 클러스터 외부	SNAT/MASQUERADE. Calico <code>natOutgoing</code> , Flannel <code>--ip-masq</code>
헤어핀	Pod가 Service를 통해 자기 자신에 도달 시 <code>0x4000</code> 마크 후 SNAT
<code>--masquerade-all</code>	모든 Service 트래픽 SNAT. 소스 IP가 소실되므로 기본값은 off

헤어핀(트래픽이 나갔던 곳으로 되돌아오는 U자 경로)은 Pod가 Service를 호출했는데 로드밸런싱 결과가 자기 자신일 때 생긴다. DNAT 후에는 출발지와 목적지가 모두 자기 Pod IP가 된다. 이대로 전달되면 Pod는 응답을 호스트로 보내지 않고 자기 안에서 바로 처리한다. 그러면 conntrack이 출발지를 ClusterIP로 되돌려놓는 역변환을 거치지 못한다. 클라이언트는 ClusterIP에서 응답이 오기를 기다리는데 Pod IP로 응답이 오니 버려진다. 그래서 kube-proxy는 이런 패킷에 `0x4000` 마크를 찍어두고 POSTROUTING에서 MASQUERADE한다. 출발지가 바뀌면 응답이 반드시 호스트를 거쳐 돌아오고, 그 과정에서 주소가 원래대로 복원된다.

- `--masquerade-all`은 이 예외를 전체로 확대한 옵션이다. 편하지만 모든 Service 트래픽의 소스 IP가 노드 IP로 덮이므로, 1.2에서 본 문제가 클러스터 안에서 재현된다. 기본값이 off인 이유다.

Calico `natOutgoing`이 적용되는 조건은 **출발지가 `natOutgoing: true`인 IPPool에 속하고, 목적지가 어떤 IPPool에도 속하지 않을 때**다. 목적지가 다른 Pod면 IPPool 안이므로 SNAT이 걸리지 않는다. "클러스터 밖으로 나갈 때만"을 이렇게 판정한다.

2. Pod 네트워크 생성 경로



2.1 Pod 생성 시퀀스

1. 선언

- `kubectl apply` 는 `apiserver`에 원하는 상태를 등록한다.
- `apiserver`는 인증·인가·admission을 거쳐 `etcd`에 기록하고 종료한다.
- 이 시점의 Pod는 어느 노드에도 존재하지 않는다.
- 단순히 `etcd`에 적힌 선언일 뿐

2. 배치

- a. 스케줄러는 `nodeName` 이 비어 있는 Pod를 watch로 감지한다.
- b. 필터링과 스코어링으로 노드를 선택한 뒤, `pod.spec.nodeName` 에 노드 이름을 기록하는 것으로 역할을 마칩
- c. 스케줄러는 Pod를 해당 노드로 전송하지 않고 배치 결과를 기록하기만 함. 실행은 다른 주체의 몫

3. 위임

- a. 각 노드의 kubelet은 nodeName이 자신인 Pod를 watch
- b. 이름이 기록되는 순간 kubelet이 이를 감지
- c. **통보를 받는 구조가 아니라 스스로 발견하는 구조** (Control Plane 구성 요소 일부가 중단되어도 클러스터가 동작하는 이유)
- d. kubelet은 컨테이너를 직접 생성하지 않고 CRI(gRPC)로 런타임에 `RunPodSandbox()`를 요청 - "Pod 샌드박스를 생성하라"라는 요청 (네트워크를 구성하라는 것이 아님)

4. 샌드박스 생성 (containerd)

- a. network namespace 생성
 - i. 생성된 netns의 소유권을 pause 컨테이너에 부여. pause는 실행 로직이 없는 컨테이너이며, 존재 목적이 netns 유지 그 자체.
 - ii. 앱 컨테이너가 종료·재시작되어도 pause가 netns를 점유하고 있으므로 Pod IP가 변경되지 않음 → CrashLoopBackOff 상태에서 반복 재시작되어도 IP가 유지되는 이유
- b. `/etc/cni/net.d/*.conf` 읽기
 - i. CNI 설정 파일을 읽는 주체는 kubelet이 아니라 **containerd**
 - ii. dockershim이 존재하던 v1.23까지는 kubelet 내부에 네트워크 플러그인 코드가 있었으나, v1.24에서 dockershim이 제거되면서 이 역할이 CRI 런타임으로 이관
- c. `/opt/cni/bin/<plugin>` 실행
 - i. gRPC나 소켓이 아니라 프로세스를 exec
 - ii. 어느 netns에, 어떤 이름으로 생성할 것인가 → 환경변수로 전달
 - iii. 네트워크 설정 → stdin의 JSON으로 전달

5. 네트워크 생성 (CNI 플러그인)

- a. veth pair 생성
 - i. 커널 내부에서 완결되는 가상 케이블
 - ii. 항상 쌍으로 생성
- b. 한쪽 끝을 Pod netns로 이동 후 `eth0` 으로 rename → Pod 내부에서 관측되는 `eth0` 가 이 시점에 생성
- c. IPAM 호출 및 주소 부여
- d. Pod 내부 라우팅 테이블 설정
- e. **호스트 쪽 끝 처리 - CNI 구현 간 차이가 대부분 여기에 집중**

CNI	호스트 쪽 veth 처리
Flannel / kindnet	<code>cni0</code> 브리지에 연결
Calico	연결하지 않음. IP도 부여하지 않고 <code>/32</code> 라우트로 처리
Cilium	<code>tc</code> / <code>tcx</code> 혹은 eBPF 프로그램 부착

6. 복귀

- a. 플러그인이 stdout으로 반환한 결과 JSON이 containerd를 거쳐 kubelet까지 전달
- b. kubelet이 CRI로 앱 컨테이너 생성을 요청하고, **런타임이** 이를 pause의 netns에 join시킴
- c. 이 구조는 같은 Pod 내 컨테이너 간 `localhost` 통신이 가능하게 하면서, 동시에 같은 포트를 중복해서 사용할 수 없게 함 (둘 다 netns를 공유하기 때문)

7. 전파

- a. kubelet이 status.podIP를 apiserver에 보고
- b. 이후 **status.podIP → EndpointSlice 컨트롤러 → kube-proxy → iptables**를 갱신
- c. EndpointSlice 컨트롤러가 해당 IP를 Service의 백엔드 목록에 추가하고, kube-proxy가 이를 watch하여 데이터 플레인 룰을 갱신
- d. Pod IP가 확정된 이후에야 Service가 해당 Pod를 지목할 수 있음. CNI가 선행하고 kube-proxy가 후행하는 순서가 여기서 결정. (두 구성 요소가 동시에 수행하지 않음)

즉 kubelet은 요청만 하고, containerd는 샌드박스과 netns를 생성하며, 실제 네트워크는 CNI 플러그인이 구성하고, kube-proxy는 그 결과 위에 Service를 엮는 구조다.

이 흐름의 특징은 어느 단계에서도 "Pod를 만들어라"는 명령이 하달되지 않는다는 점이다. 각 구성 요소는 apiserver를 watch하다 자신이 처리할 대상을 스스로 발견하고, 자기 몫만 수행한 뒤 결과를 다시 apiserver에 기록한다. 명령 전달이 아니라 **상태 관찰과 수렴**으로 동작하므로, 중간에 한 구성 요소가 죽었다 살아나도 남은 일을 이어서 처리할 수 있다.

단계	주체	하는 일
선언	apiserver	인증·인가·admission 후 etcd 기록. Pod는 아직 어느 노드에도 없다
배치	scheduler	<code>pod.spec.nodeName</code> 에 노드 이름을 기록. 전송하지 않는다
위임	kubelet	watch로 자기 Pod를 발견 → CRI 로 <code>RunPodSandbox()</code> 요청
샌드박스	containerd	netns 생성 → conflist 읽기 → CNI 바이너리 exec
네트워크	CNI plugin	veth·IPAM·라우트 설정
복귀	kubelet	앱 컨테이너를 pause의 netns에 join → <code>status.podIP</code> 보고
전파	EndpointSlice → kube-proxy	룰 갱신

2.2 CNI 규격

CNI의 계약은 "실행 파일 + 환경변수 + stdin/stdout JSON"이 전부

bash

```
CNI_COMMAND=ADD          # ADD | DEL | CHECK | GC | VERSION
| STATUS
CNI_CONTAINERID=<id>
CNI_NETNS=/var/run/netns/xxx # 이 netns 안에
CNI_IFNAME=eth0             # 이 이름으로
CNI_PATH=/opt/cni/bin
```

환경변수가 "어디에 무엇을 만들지"를, stdin JSON이 "어떤 설정으로"를 전달하고, 플러그인은 결과를 stdout JSON으로 돌려준다. 성공하면 exit code 0, 실패하면 에러 JSON이다. 흔한 리눅스 프로그램의 관례를 그대로 쓴다.

`ADD` 로 인터페이스를 만들고 `DEL` 로 정리하며, `CHECK` 로 상태를 확인한다. Pod가 사라질 때 `DEL` 이 호출되어야 IP가 반납되는데, 이 호출이 유실되면 IP가 그대로 예약된 채 남는다.

이 단순함이 CNI가 Docker의 CNM(libnetwork)을 대체한 배경이다. CNM은 Docker 데몬에 종속된 모델이라 다른 런타임이 채택하기 어려웠던 반면, CNI는 프로세스 하나만 실행할 수 있으면 붙는다. containerd, CRI-O, Podman이 동일한 플러그인 바이너리를 공유한다.

2.3 conflist 체이닝

CNI는 여러 플러그인을 순서대로 실행할 수 있다.

```
"plugins": [
  { "type": "calico",      "ipam": { "type": "calico-ipam" } },
  { "type": "portmap",    "capabilities": { "portMappings": true } },
  { "type": "bandwidth",  "capabilities": { "bandwidth": true } },
  { "type": "tuning",     "sysctl": { "net.core.somaxconn": "1024" } }
]
```

- 첫 번째 플러그인만 인터페이스를 만든다. 나머지는 앞 단계 결과(`prevResult`)를 stdin으로 받아 덧붙인다
- `DEL` 은 역순 실행

이 구조가 몇 가지를 설명한다. `hostPort` 가 동작하는 것은 `portmap` 이 DNAT 룰을 추가하기 때문이고, Pod 대역폭 제한은 `bandwidth` 가 tc qdisc를 설정하기 때문이며, `securityContext.sysctls` 는 `tuning` 이 처리한다. Multus는 이 체이닝을 확장한 형태다.

2.4 CNI 상태와 노드 컨디션

```
containerd: CNI config 로드 실패 → lastCNILoadStatus 기록
↓
kubelet: CRI Status() RPC
RuntimeStatus.conditions[NetworkReady] = false (NetworkPluginNotReady)
↓
노드 Ready=False
```

```
KubeletNotReady container runtime network not ready:
NetworkReady=false reason:NetworkPluginNotReady
message:Network plugin returns error: cni plugin not initialized
```


네트워크 플러그인 하나 때문에 **노드 전체가 NotReady가 되는 것**이 과해 보이지만 타당하다. CNI가 없으면 어떤 Pod도 IP를 받을 수 없으므로, 그 노드는 스케줄을 받아봐야 전부 실패한다. 아예 배치 대상에서 빼는 편이 낫다. 새 클러스터에서 CNI를 설치하기 전까지 노드가 계속 NotReady인 것도 정상 동작이다.

진단할 때 방향을 헛갈리기 쉬운데, kubelet은 `/etc/cni/net.d`를 직접 확인하지 않고 CRI를 통해 런타임에 질의한다. **증상은 kubelet 로그에 뜨지만 원인은 런타임 쪽에 있다.** 그래서 진단도 런타임 쪽에서 수행한다.

```
crictl info | jq '.status.conditions'      # NetworkReady 조건
crictl info | jq '.lastCNILoadStatus'
journalctl -u containerd | grep -i cni
```

3. Pod IP 출처

3.1 배정 주체는 구현마다 다르다

"Pod IP는 CNI가 준다"는 서술은 절반만 맞다. CNI 플러그인이 인터페이스에 주소를 부여하는 것은 맞지만, 그 주소를 어디서 확보하는지는 구현마다 완전히 다르다.

구현	배정 주체	저장 위치
Flannel / kindnet	kube-controller-manager	<code>node.spec.podCIDR</code>
Calico	Calico IPAM (자체) — <code>podCIDR</code> 를 무시	IPAMBlock / BlockAffinity CRD
AWS VPC CNI	VPC 서브넷에서 ENI 보조 IP 직접 할당	Pod CIDR 개념 없음

Calico가 `podCIDR`를 무시한다는 점이 특히 혼란을 부른다. `kubectl get node -o yaml`에 `podCIDR` 필드가 멀쩡히 채워져 있는데 실제 Pod IP는 전혀 다른 대역에서 나온다. 이 필드는 controller-manager가 채운 것이고 Calico는 자기 IPPool을 볼 뿐이라서 그렇다. **디버깅할 때 `podCIDR`를 근거로 삼으면 안 된다.**

```
--allocate-node-cidrs=true --cluster-cidr=10.244.0.0/16 --node-cidr-mask-size=24
↓
node-1.spec.podCIDR = 10.244.0.0/24
node-2.spec.podCIDR = 10.244.1.0/24
```

노드가 조인하는 순간 `/24` 하나가 통째로 배정된다. Pod를 하나만 띄우든 200개를 띄우든 그 `/24` 는 해당 노드의 것이다. 쓰지 않아도 회수되지 않고, 모자라도 늘어나지 않는다.

3.2 IPAM

IPAM은 별도 플러그인이다. `conflist`에서 `ipam.type` 으로 지정하면 메인 플러그인이 그 바이너리를 다시 호출한다. 그래서 "네트워크를 어떻게 연결할지"와 "주소를 어디서 가져올지"를 따로 갈아끼울 수 있다.

type	방식
<code>host-local</code>	노드 로컬 디스크에 파일로 할당 기록. 순차 할당
<code>static</code>	고정 IP 지정
<code>dhcp</code>	DHCP 서버에서 임대 (별도 데몬 필요)
<code>calico-ipam</code>	IPPool → IPAMBlock(/26) → BlockAffinity

실질적으로 쓰이는 것은 `host-local` 과 CNI별 전용 IPAM이다. `dhcp` 는 별도 데몬을 상주시켜야 해서 드물고, `static` 은 테스트용에 가깝다. 아래에서는 성격이 정반대인 `host-local` 과 `calico-ipam` 을 비교한다.

3.3 host-local

플러그인을 세 번 호출한 뒤 디스크를 확인했다.

```
$ ls -l /var/lib/cni/networks/mynet/
10.22.0.2
10.22.0.3
10.22.0.4
last_reserved_ip.0
lock

$ cat /var/lib/cni/networks/mynet/10.22.0.2
demo-container-0001
eth0
```

파일명이 할당된 IP이고, 내용이 컨테이너 ID와 인터페이스 이름이다. 데이터베이스도 `etcd`도 아닌 파일시스템 자체가 상태 저장소다. `lock` 파일이 동시 할당을 막고, `last_reserved_ip.0` 이 다음 탐색 시작점을 기억한다. 구현 전체가 파일 몇 개로 끝난다. 순차 할당과 반납:

```
container-0001 → 10.22.0.2/24
container-0002 → 10.22.0.3/24
container-0003 → 10.22.0.4/24
last_reserved_ip.0 = 10.22.0.4
```

```
# CNI_COMMAND=DEL 로 container-0002 반납 후
10.22.0.2
10.22.0.4      ← 10.22.0.3 이 사라짐
```

이 구조에서 따라오는 사실이 있다. **kubelet이 비정상 종료해 DEL이 호출되지 않으면 파일이 잔존한다.** 해당 IP는 실제로 사용되지 않으면서 영구히 예약 상태로 남는다. 노드를 재부팅해도 파일은 디스크에 그대로 있으므로 저절로 낫지 않는다.

증상은 서서히 나타난다. Pod가 뜨고 지기를 반복할수록 쓸 수 없는 주소가 쌓이고, 결국 `failed to allocate for range 0: no IP addresses available in range` 로 Pod 생성이 실패한다. 노드에 실제로 떠 있는 Pod 수와 디렉터리의 파일 수가 어긋나 있으면 이 상황이다. 해결책은 디렉터리를 직접 정리하는 것뿐이다.

3.4 Calico IPAM

```
IPPool (192.168.0.0/16, blockSize: 26)
├── IPAMBlock (/26 = 64개 주소)      ← 필요할 때 claim
│   └── BlockAffinity (블록 ↔ 노드 결합)
│       └── 개별 /32 (블록 내 비트맵)
```

노드에 대역을 미리 주지 않는다. 첫 Pod가 스케줄되는 시점에 노드가 풀에서 블록 하나를 claim하고, 소진되면 추가로 claim한다. 한 노드가 불연속적인 `/26` 여러 개를 보유할 수 있다.

블록이라는 중간 단위를 두는 이유는 **경합을 줄이기 위해서**다. IP 하나를 할당할 때마다 apiserver에서 전역 락을 잡으면 Pod를 대량으로 띄울 때 병목이 된다. 대신 `/26` 단위로 한 번 확보해두고, 그 안에서는 노드가 비트맵으로 혼자 처리한다. 64개를 다 쓸 때까지 다시 조율할 필요가 없다.

- ① 내 affine 블록에서 할당 시도
- ② 없으면 새 블록 claim (기본 최대 20개)
- ③ 그래도 없으면 다른 노드 블록에서 개별 주소를 빌린다 (borrowing)
 - ※ strictAffinity: true 면 ③ 없이 실패

기본 상한이 노드당 20블록이므로 **노드당 최대 1280개 주소**다. 이 값은 공식 레퍼런스 문서에는 없고 [libcalico-go/lib/ipam/ipam.go](https://github.com/libcalico-go/lib/ipam/ipam.go)에만 존재한다. (버전별 상이)

3.5 두 모델 비교

항목	host-local + podCIDR	Calico IPAM
할당 단위	노드당 <code>/24</code> 하나, 미리 배정	<code>/26</code> 블록, 필요 시 요청

항목	host-local + podCIDR	Calico IPAM
노드당 블록	1개, 노드 수명 내내 고정	여러 개, 불연속, 증가 가능
노드 용량 상한	하드 254개 (실제로는 kubelet maxPods 기본 110이 먼저 걸림)	소프트 (블록 추가 → borrowing)
고갈 시	즉시 실패	다른 노드 블록에서 차용
주소 효율	낮음 — Pod 5개 노드도 <code>/24</code> 소비	높음
클러스터 상한	<code>/16 ÷ /24 = 256 노드</code>	<code>/16 ÷ /26 = 1024 블록</code>
상태 위치	노드 로컬 파일	CRD (분산)
Pool별 정책	없음	<code>natOutgoing</code> , <code>ipipMode</code> , <code>nodeSelector</code>
고정 IP	불가	<code>cni.projectcalico.org/ipAddrs</code>

`/16` 클러스터에서 host-local을 쓰면 노드 256개가 끝이고, 대역을 바꾸려면 클러스터를 다시 만들어야 한다. 초기에 설정한 `--cluster-cidr` 하나가 클러스터 수명 전체의 규모 상한을 결정한다.

host-local은 미리 넉넉히 배정하고 하드 상한을 감수하는 단순·무상태 모델, Calico IPAM은 필요한 만큼 요청하고 부족하면 차용하는 조밀·탄력 모델이다. 후자의 대가는 분산 할당자와 CRD 상태, 그리고 borrowing의 부작용이다.

borrowing의 대가: 차용한 주소는 차용 노드가 광고하는 aggregate 밖에 있다. 원래 소유 노드의 `/26` 라우트가 소유자를 가리키므로, Calico는 차용 주소에 대해 개별 `/32` 호스트 라우트를 별도 광고해야 한다. 라우팅 테이블이 팽창하며, BGP로 물리 패브릭과 피어링하는 환경에서는 그 `/32` 가 물리 네트워크까지 전파된다. `blockSize` 는 기존 pool에서 변경할 수 없다.

Q&A

Q1. ClusterIP는 어떤 NIC에도 바인딩되어 있지 않는데 Pod는 어떻게 그 주소에 도달하는가

Service CIDR이 가상이기에 소유한 인터페이스가 없어 ARP를 해도 응답할 주체가 없다.

애초에 ARP를 하지 않기에 도달이 가능하다. `10.96.0.100` 은 Pod의 서브넷 밖이므로 Pod 입장에서는 외부로 나가는 패킷이다. 이런 경우 커널은 목적지의 MAC이 아니라 **default gateway의 MAC**을 찾는다. 게이트웨이는 호스트이고 실재하므로 ARP가 정상적으로 성립한다.

즉 Pod는 "이 주소가 어디 있는지"를 알 필요가 없다. 모르는 주소는 게이트웨이에게 넘긴다는 일반 규칙만 따르면 된다. 목적지 IP는 10.96.0.100 인 채로 유지되고, 실제 변환은 호스트의 netfilter PREROUTING에서 DNAT으로 일어난다.

이 구조가 kube-proxy 모드 간 차이도 설명한다. iptables 모드는 PREROUTING(라우팅 결정 전)에서 DNAT하므로 ClusterIP가 로컬 주소일 필요가 없다. IPVS 모드는 LOCAL_IN 훅에서 동작하므로 패킷이 라우팅 결정에서 "내 것"으로 판정되어야 한다. 그래서 kube-proxy가 kube-ipvs0 더미 인터페이스를 생성하고 모든 ClusterIP를 바인딩한다.

```
$ ip addr show kube-ipvs0
kube-ipvs0: <BROADCAST,NOARP> mtu 1500 qdisc noop state DOWN
    inet 10.96.0.1/32 scope global kube-ipvs0
    inet 10.96.0.10/32 scope global kube-ipvs0
```

NOARP 플래그와 state DOWN 이 이 인터페이스의 성격을 보여준다. 실제로 패킷을 주고받지 않고 ARP 응답도 하지 않는다. 오직 커널의 로컬 주소 목록에 등록되기 위해서만 존재한다.

Q2. CNI 플러그인은 실행되고 바로 종료하는 프로세스인데, 할당 상태는 어디에 남는가

ADD 한 번 실행되고 끝난다. 메모리에 아무것도 들고 있을 수 없다. 그런데 "이 IP는 이미 누가 쓰고 있다"는 사실은 다음 호출 때 반드시 알아야 한다. 상태를 프로세스 밖에 둘 수밖에 없고, 어디에 두느냐가 IPAM 구현을 가른다.

IPAM	상태 위치	성질
host-local	노드 로컬 파일 /var/lib/cni/networks/<net>/	노드 안에서만 유효. 락도 파일
calico-ipam	apiserver의 CRD (IPAMBlock / BlockAffinity)	클러스터 전역 공유. 노드 간 조율 가능
AWS VPC CNI	VPC (클라우드 컨트롤 플레인)	상태가 쿠버네티스 밖에 있음

host-local은 자기 노드의 파일만 보므로 옆 노드에 남는 주소가 있는지 알 방법이 없다. 그래서 고갈되면 그냥 실패한다. 반대로 Calico는 상태가 apiserver에 있으니 다른 노드의 블록을 조회하고 빌려올 수 있다. borrowing이 가능한 이유는 알고리즘이 똑똑해서가 아니라 상태를 공유하고 있기 때문이다. 대가로 IP를 할당할 때마다 apiserver를 거치는 비용을 낸다.

상태를 지우는 주체가 DEL 호출뿐인데, 노드나 kubelet이 비정상 종료하면 그 호출이 일어나지 않는다. 아무도 "이 상태가 유효한지" 되묻지 않으므로 잘못된 상태가 스스로 고쳐지지 않는다. 쿠버네티스 대부분이 상태를 관찰하고 수렴시키는 방식으로 동작하는 것과 대조적이다.

이 빈틈을 메우려는 시도가 CNI 규격의 `GC` 커맨드다. 런타임이 "현재 살아 있는 컨테이너 목록"을 넘기면 플러그인이 그 밖의 자원을 정리한다. 관찰과 수렴 모델을 IPAM에도 도입하려는 것이다. Calico 쪽에서는 `calicoctl ipam check` 로 실제 Pod와 할당 기록의 불일치를 찾아낼 수 있다.